

Tech Stadium

株式会社 クリーク・アンド・リバー社 Tech Stadium



アジェンダ

- はじめに
- 外部のメンバ関数の呼び出し方法
- DelegateとLambda式
- UIのレイアウトグループ
- UIのスクロールビュー

解説

- 外部のメンバ関数の呼び出し方法
- DelegateとLambda式
- 機能の説明



はじめに

本日の授業では、

Unityで多く使われる機能とUIについて学習していきます。

前半は、Unityで多く使われる技術とUIの説明を行います。

後半は、Projectの解説を行います。



はじめに

講義に使用するスライドやプロジェクトデータ

リモートのリポジトリパス

[TechStadium_Unity-ItemList] https://gitlab.com/creektech/techstadium_unity-itemlist.git

Curriculum スライドが格納されている

UnityProject Unityサンプルプロジェクトが格納されている

以上



アジェンダ

- はじめに
- 外部のメンバ関数の呼び出し方法
- DelegateとLambda式
- UIのレイアウトグループ
- UIのスクロールビュー

解説

- 外部のメンバ関数の呼び出し方法
- DelegateとLambda式
- 機能の説明



外部のメンバ関数の呼び出し方法

- 外部のクラスのメンバ関数や変数にアクセスするときには、まずはそのクラスにアクセスする必要があります。そのクラスにアクセスする方法として、基本的にはそのクラスが紐づいたGameObjectを経由してクラスを呼び出すことが多いですが、MonoBehaviorを継承していない場合等でGameObjectに紐づいていないクラスにアクセスしたい場合もあると思います。
次にGameObjectを経由してクラスにアクセスする方法3つと、GameObjectを経由せずクラスにアクセスする方法2つを説明します。

GameObjectにGetComponent<>を利用して該当するClassにアクセスする方法

- SerializeFieldでGameObjectにアクセスする方法
- GameObject.FindでGameObjectにアクセスする方法
- InstantiateでReturnされたGameObjectにアクセスする方法

GameObjectを経由せずClassにアクセスする方法

- Staticを利用してアクセスする方法
- Singleton Patternでアクセスする方法

この5つの方法を利用して、他のClassの変数や関数にアクセスします。



外部のメンバ関数の呼び出し方法

1. SerializeFieldでGameObjectにアクセスする方法

条件 : 自分自身と対象のオブジェクトがHierarchyに存在しなければなりません。

この場合にはGameObjectのスクリプトに[SerializeField]と宣言して、Unity Editor上でアタッチすることでアクセスすることができます。

2. GameObject.FindでGameObjectにアクセスする方法

対象がPrefabである場合には、Prefabが実体化された後に取得しなければなりません。
実体化されたPrefabにはGameObject.Findで名前を指定してアクセスすることができます。

ただし、GameObject.FindはHierarchy全てを検索するので処理コストが高くなってしまい、あまり使われません。



外部のメンバ関数の呼び出し方法

SerializeFieldを使う方法

The diagram illustrates the process in three numbered steps:

- 1**: A C# script snippet is shown with `[SerializeField]` attributes on `private Button[] _funcButtons = default;`, `private GameObject _itemPrefab = default;`, and `private GameObject _content = default; //ボタン`. Below the code, the Unity Hierarchy window shows the **Item List Manager (Script)** attached to a GameObject. The **Func Buttons** section is expanded, and `Item Prefab` is assigned to `ButtonItem`.
- 2**: A red arrow points from the `_itemPrefab` field in the script to the `Instantiate` method in the code block on the right.
- 3**: A red arrow points from the `_content` field in the script to the `GetComponent` and `Add` methods in the code block on the right.

The code block on the right contains the following C# code:

```
//生成
GameObject itemObject = Instantiate(_itemPrefab, Vector3.zero, Quaternion.identity);
itemObject.transform.SetParent(_content.transform);

//リスト追加
ItemContents item_content = itemObject.GetComponent<ItemContents>();
_itemButtons.Add(item_content);
```

上のプロジェクトのように

1. スクリプトにGameObjectをアタッチする
2. アタッチしたGameObjectを外部のスクリプトをGetComponent利用して呼び出す。
3. 呼び出したスクリプト内の変数/関数にアクセス

の順でアクセスすることが出来ます。

外部のメンバ関数の呼び出し方法

3. InstantiateでReturnされたGameObjectにアクセスする

InstantiateしたObjectにReturn Valueを利用する方法があります。

実体化させたいオブジェクトをInstantiateの引数として渡すことでクローンが作成され、戻り値のGameObjectにアクセスし、GetComponent<>することでClassにアクセスすることができます。



外部のメンバ関数の呼び出し方法

4. Staticを利用してアクセスする方法

変数や関数をStatic化すればインスタンスを生成せずとも、どのクラスからでも直接参照できますが、値の変更や再利用が難しくなるという難点があります。

5. Singleton Patternでアクセスする方法

Staticを利用すると発生する難点が多いため最近ではStaticより柔軟な対応が可能なSingleton Patternがよく使われます。

GameObjectを使わずにClassにアクセスする方法として、Singleton Patternがあります。

まずは、Singleton化したいClassにPrivate型のInstanceを変数に作ります。

Instance を返す関数をStatic化することで他のClassからはInstanceを経由することでしかアクセスできなくなります。

Class自体を変更せずアクセスすることができる方法です。



アジェンダ

- はじめに
- 外部のメンバ関数の呼び出し方法
- DelegateとLambda式
- UIのレイアウトグループ
- UIのスクロールビュー

解説

- 外部のメンバ関数の呼び出し方法
- DelegateとLambda式
- 機能の説明



Delegateとは

- Delegateは代理人という意味で、関数を自分で定義したデリゲート型の変数に格納して、変数として扱うことができるものです。Callbackメソッドを実装するときによく使われます。

```
public delegate int Sampledelegate(int samplenum1, int samplenum2); // Delegateの宣言とDelegateの戻り値や引数を定める
void Start()
{
    _showsampletext.text = "";
    Sampledelegate sampledel;
    sampledel = Plus;
    Calculator(37, 12, sampledel);

    sampledel = Sub;
    Calculator(37, 12, sampledel);
}

public int Plus(int number01, int number02) //数字を二つ入れて加算
{
    return number01 + number02; //数字1番と2番を加算してReturnします。
}

public int Sub(int number01, int number02) // 数字を二つ入れて減算
{
    return number01 - number02; //数字1番と2番を減算してReturnします。
}

public void Calculator(int firstnum, int secondnum, Sampledelegate calcFunction) //受け取った数字を計算してTextlに表示します
{
    _showsampletext.text += "Result = " + calcFunction(firstnum, secondnum) + "¥n";
}
```

実行結果

Result = 49

Result = 25

Lambda式とは

- Lambda式は無名の関数を簡略的に定義し、可読性を上げるために使用します。

(入力するパラメータ) => { 実行する命令文 } //このような形で使います。

() => Write("No"); // 入力パラメータが無い場合

// 入力パラメータが1～2ある場合

(print) => Write(print);

(sample, example) => { Write(sample); Write(example); }

// 入力パラメータのタイプを明示する場合

(string sample, int number) => Write(sample, number);

() => { Function(); } //関数を入れる時

参考URL

C# Lambda式とは <https://csharp.keicode.com/basic/lambda-expressions.php>



アジェンダ

- はじめに
- 外部のメンバ関数の呼び出し方法
- DelegateとLambda式
- UIのレイアウトグループ
- UIのスクロールビュー

解説

- 外部のメンバ関数の呼び出し方法
- DelegateとLambda式
- 機能の説明



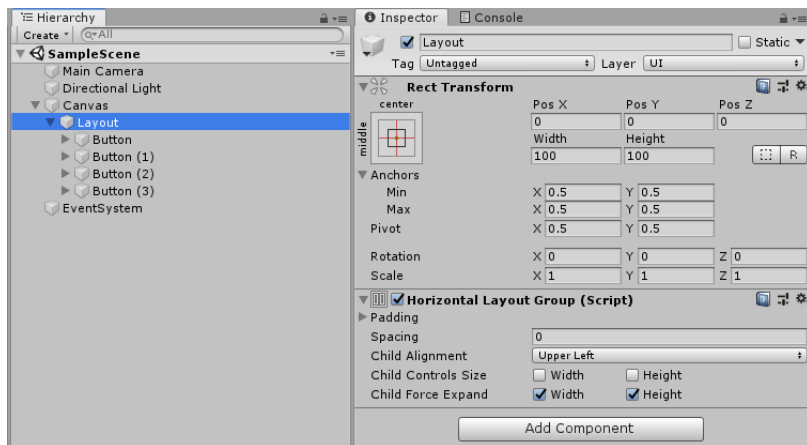
UI Layout Groupとは

- 子のUI要素をきれいに並べるためのコンポーネント
- ボタンなどを等間隔で並べたい場合、レイアウトグループを使うことで簡単にUIを並べることが可能
- レイアウトグループの種類
 - Horizontal Layout Group
子のUIを水平に並べるグループ
 - Vertical Layout Group
子のUIを垂直方向に並べるグループ
 - Grid Layout Group
子のUIをグリッド状態に並べるグループ

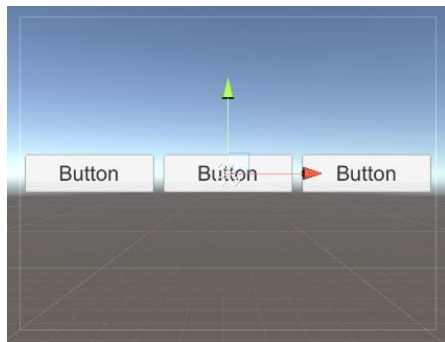


UI Layout Groupの導入

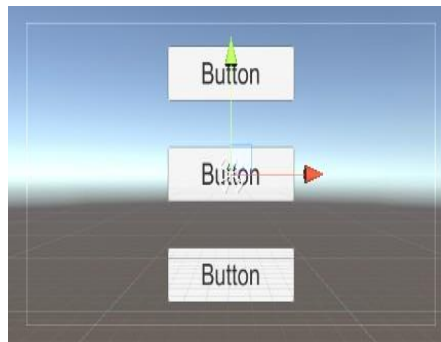
- UnityのプロジェクトにCanvasを作ってCanvasを右クリックして、Create Emptyを押した後、作られたGame Objectの名前をLayoutとする。
- LayoutでAdd Componentを押して、Horizon、Vertical、Grid Layout Groupの中のどれかを追加する。
- 追加したLayout Groupを右クリックし、UIのButtonを押して、ボタンを好きなだけ追加する。



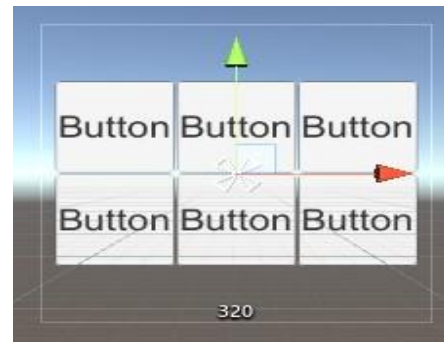
UI Layout Groupの例



Horizontal Layout Group



Vertical Layout Group



Grid Layout Group

アジェンダ

- はじめに
- 外部のメンバ関数の呼び出し方法
- DelegateとLambda式
- UIのレイアウトグループ
- UIのスクロールビュー

解説

- 外部のメンバ関数の呼び出し方法
- DelegateとLambda式
- 機能の説明



UIのスクロールビューとは

- 子のスクロール要素を綺麗に並べるためのコンポーネント
- UIのスクロールバーを簡単に並べることが可能



UIのスクロールビューの導入

- UnityのプロジェクトにCanvasを作ってCanvasを右クリックして、UIのScrollViewを作成する。
- ScrollViewのScroll Rectで、HorizontalとVerticalの使わないどちらかのチェックを解除する。
- ScrollViewのScroll Rectで、Movement TypeをClampedに変更する。
- ScrollViewのContentのチェックされたHorizontal、またはVertical LayoutとContent Size FilterをAdd Componentを押して追加する。
- Contentの下に、イメージやボタンなどを好きなだけ追加する。
- Contentの下に作られたオブジェクト各々にAdd Componentを押してLayout Elementを追加する。

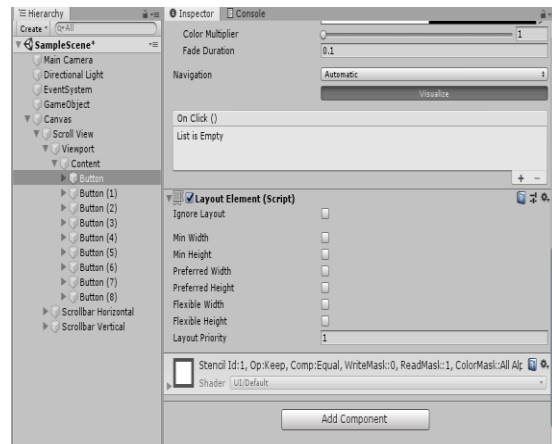
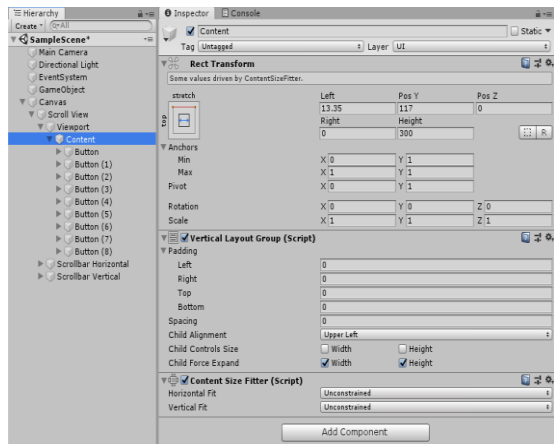
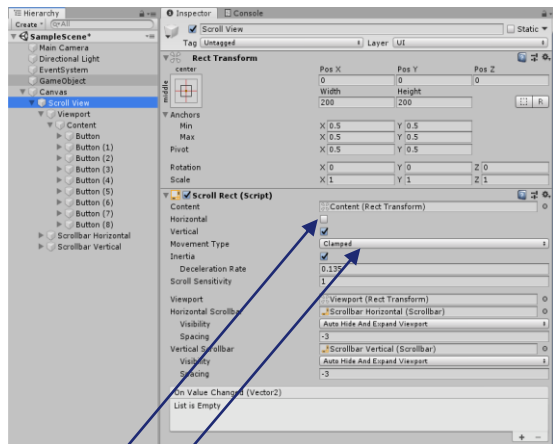


UIのスクロールビューの導入

- Content : Scrollの中身に入れる内容物
- Viewport : Scrollの内部で表示される画面
- Content Size Fitter: Contentのサイズを調整、Unconstrainedは手動でサイズを調整可能、Preferred Sizeは手動でサイズを調整不可能、自動でサイズが調整される
- Layout Element: Contentの内部で追加されるオブジェクトの各々要素をコントロール可能にする
 - ・Ignore Layout: 上位のLayoutを無視する
 - ・Min: 分けられたサイズだけ調整される
 - ・Flexible: Minのサイズより空間が余った場合、Preferredにそのサイズを加え、さらに空間が余った場合はFlexibleのサイズに加えられる。Flexibleのサイズは0.0001～1

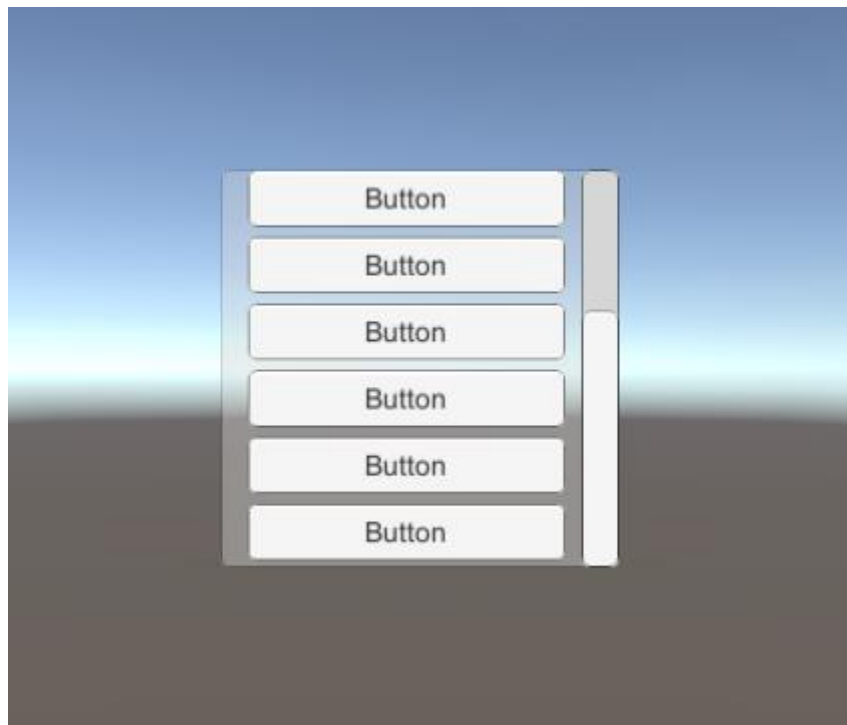


UIのスクロールビューの導入



- HorizontalとVerticalの使わない方をチェックを解除
- Movement TypeをClampedに変更

UIのスクロールビューの例



アジェンダ

- はじめに
- 外部のメンバ関数の呼び出し方法
- DelegateとLambda式
- UIのレイアウトグループ
- UIのスクロールビュー

解説

- 外部のメンバ関数の呼び出し方法
- DelegateとLambda式
- 機能の説明



外部のメンバ関数の呼び出し方法

```
[SerializeField] private Button[] _funcButtons = default;
[SerializeField] private GameObject _itemPrefab = default;
[SerializeField] private GameObject _content = default; // ボタン生成場所
[SerializeField] private Text _itemInfoText = default; // ボタンの情報を表示するテキスト
private List<ItemContents> _itemButtons; // ボタンリスト
```

```
// 生成
GameObject itemObject = Instantiate(_itemPrefab, Vector3.zero, Quaternion.identity);
itemObject.transform.SetParent(_content.transform);

// リスト追加
ItemContents item_content = itemObject.GetComponent<ItemContents>();
_itemButtons.Add(item_content);

item_content.SendIdToItem(itemId); // アイテム識別Idを与える
item_content.SendOnClickDelegate(OnClickItemButton); // アイテムにアイテムボタンが押された時に
```

1. [SerializeField]でPrefabを設定し、それを基にInstantiate()でPrefabObjectを生成します。

2. GameObjectの戻り値にGetComponentをしてItemContentsスクリプトを取得します。

3. 取得した外部のスクリプトのメンバー関数を呼び出して値を入れます。



アジェンダ

- はじめに
- 外部のメンバ関数の呼び出し方法
- DelegateとLambda式
- UIのレイアウトグループ
- UIのスクロールビュー

解説

- 外部のメンバ関数の呼び出し方法
- DelegateとLambda式
- 機能の説明



Delegate と Lambda式

```
public delegate void OnDelegate(int num, ItemContents itemcontents); //delegate定義
private int _itemId = 0; //Item識別Id
private OnDelegate _delegateOnClickButton = null; //デリゲート変数
```

```
/// <summary>
/// デリゲート変数に、ItemListManagerから受け取った関数を代入
/// </summary>
/// <param name="del">ItemListManagerから受け取った関数</param>
public void SendOnClickDelegate(OnDelegate del)
{
    _delegateOnClickButton = del;
}
```

```
//自身(Item)の表示を設定
_thisBtnTxt.text = "Item : " + _itemId;
_selfBtn.onClick.AddListener(() =>
{
    _delegateOnClickButton(_itemId, this);
});
```

1. Delegateを宣言し、Delegateの形を決めます。
その後、Delegate変数を宣言して初期化します。
2. ItemListManagerClassから渡した関数をDelegate変数に代入します。
3. onClick.AddListenerで呼び出される処理をLambda式で実装しています。
4. Delegate変数の引数に itemId と ItemContents Class自身を指定することでItemListManager Classにクリックされたアイテムの情報を渡しています。

アジェンダ

- はじめに
- 外部のメンバ関数の呼び出し方法
- DelegateとLambda式
- UIのレイアウトグループ
- UIのスクロールビュー

解説

- 外部のメンバ関数の呼び出し方法
- DelegateとLambda式
- 機能の解説



アイテム生成

```
[SerializeField] private GameObject _itemPrefab = default;
[SerializeField] private GameObject _content = default; //ボタン生成場所
private List<ItemContents> _itemButtons;

private void MakeButton(int itemId)
{
    //生成
    GameObject itemObject = Instantiate(_itemPrefab, Vector3.zero, Quaternion.identity);
    itemObject.transform.SetParent(_content.transform);

    //リスト追加
    ItemContents item_content = itemObject.GetComponent<ItemContents>();
    _itemButtons.Add(item_content);
}
```

- ・[SerializeField]でアイテムPrefabを指定し、それを基にInstantiate()でオブジェクトを生成します

- ・アイテムはSetParent()を用いてScrollViewのcontentの下に移動させます

- ・生成したオブジェクトから、GetComponentを用いてItemContentsクラスを取得し、ItemContents型のリストで管理できるようにします。

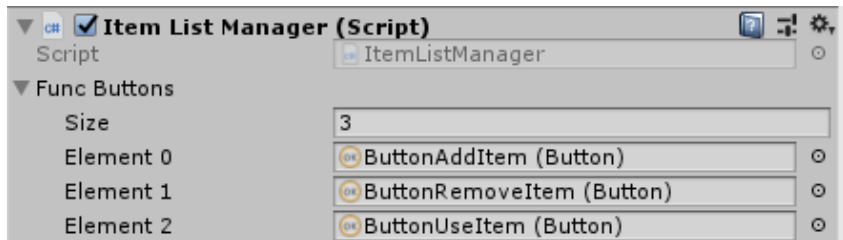


Scene上のボタンUIにスクリプトからアクセス

```
private enum BTN
{
    Add = 0,
    Remove,
    Use
}

[SerializeField] private Button[] _funcButtons = default;

void Start()
{
    //button_ItemAdd
    _funcButtons[(int)BTN.Add].onClick.AddListener(OnClickAddItemButton);
    //button_ItemRemove
    _funcButtons[(int)BTN.Remove].onClick.AddListener(OnClickRemoveItemButton);
    //button_ItemUse
    _funcButtons[(int)BTN.Use].onClick.AddListener(OnClickUseItemButton);
    _funcButtons[(int)BTN.Use].interactable = false;
}
```



- ・[SerializeField]でボタン型の配列を定義すると、Inspector上で好きな数のUIのボタンをアタッチすることが出来ます。

- ・定義したボタン型の配列変数[要素番号]とすることで、該当するボタンを扱えるようになります。

- ・enum型でそれぞれのボタンの名前を決めておくことで、要素番号を指定する代わりに自分で指定した名前を使用することが出来るようになります。

参考: <https://www.sejuku.net/blog/55897>



enum型を用いる利点

- Button[0]、Button[1]…よりも、Button[BTN.Add]やButton[BTN.Remove]と命名したほうが分かりやすいです。AddボタンとRemoveボタンの間にボタンを一つ追加したい、となった場合でもボタンの識別がしやすいためミスが少なくなります。
- マジックナンバーを使わなくすることが出来ます。

マジックナンバーとは？

- プログラムに直接記述された値のことですが、その値だけではなにを意味するのか分かりづらいです。またその値を変更するとなったときに、その値が複数の個所で使われている場合、それらすべてを探して変更しなければなりません。こういった理由により、マジックナンバーは使うべきではないとされています。

アイテムの追加処理

```
private void OnClickAddItemButton()
{
    int itemPickup = Random.Range(0, DEFAULT_ITEM_SIZE);
    MakeButton(itemPickup);

    //Removeボタン有効化
    _funcButtons[(int)BTN.Remove].interactable = true;
}
```

```
private void MakeButton(int itemId)
{
    //生成
    GameObject itemObject = Instantiate(_itemPrefab, Vector3.zero, Quaternion.identity);
    itemObject.transform.SetParent(_content.transform);

    //リスト追加
    ItemContents item_content = itemObject.GetComponent<ItemContents>();
    _itemButtons.Add(item_content);

    // アイテム識別Idを与える
    item_content.SendIdToItem(itemId);
    // アイテムにアイテムボタンが押された時に実行してほしい関数を伝える=デリゲート
    item_content.SendOnClickDelegate(OnClickItemButton);
}
```

Add Item

AddItemボタンでアイテム追加。

AddItemボタンが押されたときに呼ばれる
OnClickAddItemButton関数では、
0～定数のランダムな数字を引数に、
MakeButton関数でアイテムを生成しています。

・ MakeButton関数では

- ①アイテムの生成
- ②生成したアイテムをリストに追加
- ③生成された各オブジェクトが持つ
ItemContentsクラスのメンバ変数に値を代入
を行っています。

アイテムの削除処理

Remove Item

Remove Itemボタンで
一番下にあるアイテムを削除

```
private void OnClickRemoveItemButton()
{
    if (_itemButtons.Count > 0)
    {
        // _itemButtonsの要素数をカウントし、最後のアイテムを削除
        RemoveItem(_itemButtons.Count - 1);
    }
}
```

```
private void RemoveItem(int targetNum)
{
    Destroy(_itemButtons[targetNum].gameObject);
    _itemButtons.RemoveAt(targetNum);
    // UseButton無効化
    _funcButtons[(int)BTN.Use].interactable = false;
    if (_itemButtons.Count <= 0)
    {
        _funcButtons[(int)BTN.Remove].interactable = false;
    }
}
```

・Removeボタン押下時にはOnClickremoveButton()が呼び出されます。

・関数内でアイテムリストの要素数をカウントし、最後のアイテムの要素番号を取得します。

・Remove関数は、引数に削除させたいターゲットの要素番号を受け取ります。

・Remove関数の引数にカウント-1の値を渡すことで、アイテムリストの最後にあるアイテムが削除されます。

アイテムの削除処理

Use Item

Use Itemボタンで
選択したアイテムを削除

```
private void OnClickUseItemButton()
{
    RemoveCurrentItem();
}
```

```
private void RemoveCurrentItem()
{
    int contentNumber = GetItemTargetNum(_current_ItemContents);
    if(contentNumber >= 0)
    {
        RemoveItem(contentNumber);
    }
}
```

```
private int GetItemTargetNum(ItemContents target)
{
    for (int index = 0; index < _itemButtons.Count; index++)
    {
        if (target == _itemButtons[index])
        {
            return index;
        }
    }
    return -1;
}
```

RemoveItem(int targetNum) 呼び出し

UseItemボタン押下時は、OnClickUseItemButton()が呼び出されます。

呼び出されたRemoveCurrentItem()では、削除対象のアイテムをItemContentsクラスから受け取り、GetTargetNum()に渡します。

GetTargetNum()は、該当するアイテムの要素番号を見つけ、RemoveItem()に要素番号を伝えます。

RemoveItemはターゲットのアイテムを削除します。

共通のアイテムの削除処理 – RemoveItem()

```
private void RemoveItem(int targetNum)
{
    Destroy(_itemButtons[targetNum].gameObject);
    _itemButtons.RemoveAt(targetNum);
    //UseButton無効化
    _funcButtons[(int)BTN.Use].interactable = false;
    if(_itemButtons.Count <= 0)
    {
        _funcButtons[(int)BTN.Remove].interactable = false;
    }
}
```

- ・ 削除する対象となるアイテムの識別Idを引数で受け取り、アイテムを削除します。
- ・ アイテムを管理しているリストの中から識別Idを用いて削除すべきアイテムを特定しています。
- ・ 削除するのはアイテム本体のGameObjectだけではなく、アイテムを管理しているリストの中からも該当の要素を削除しなければなりません。
- ・ Useボタンはアイテムの削除処理が行われるごとに無効化し、Removeボタンはリストの要素数が0以下になったときに無効化されます。

関数の共通化

UseItemボタンからアイテムを削除する場合、
RemoveItemボタンからアイテムを削除する場合、どちらもRemove()を呼んでいます。

処理が共通している場合は、その処理を持たせた関数を作成するほうが良いです。
一つの関数を共有することでプログラムの構成も簡素になりますし、
処理に変更を加えたい場合にも修正が必要な範囲が狭くなるため、ミスが少なくなります。



課題

アイテムごとに情報を持たせ、それらを利用して何らかの処理を実装してください。

(例: アイテムにIDや名前を持たせて、アイテム選択時にそれらを表示させる)

(例: プレイヤー情報として所持金を設定しておき、
アイテムにも値段を設定してそれを買えるかどうかの判定を行う)



Q&A

